

Mechanistic Analysis of Architecture and Semantic-ID Design in Generative Recommendation

Project Brief

1 Primary Goal

The goal of this project is to understand how internal computation of Semantic ID based recommendation systems is organized. We study this through three axes: recommendation paradigm (autoregressive, diffusion), SID quantization method (RQ-VAE, RQ-Kmeans, Parallel), and scaling parameters like codebook size and number of digits in a SID. We plan to trace the type of information being represented, when that information is used, and how it causally affects the final generated recommendation.

The closest interpretability work applies sparse autoencoders (SAEs) to the hidden states of conventional sequential recommenders or to static recommendation embeddings [1], [2], [3]. Our setting instead compares models that generate multi-digit SIDs. [4], [5] apply SAEs to recommendation embeddings (SAE training data is item embedding table).

2 Research Questions We Ask

RQ1: How does the recommendation paradigm shape internal computation? Keeping the SID representation fixed, do autoregressive and diffusion decoders extract and causally use the same information from user history? What evidence is used at each generation or denoising step? Does diffusion learn a non-trivial, user-dependent digit-position policy, or does it repeatedly follow an AR-like order? Are causally validated feature families shared, aligned, or specific to one paradigm?

RQ2: What information is organized by the SID construction and its digits? Keeping the recommendation paradigm fixed, how do RQ-VAE, RQ-KMeans, and parallel quantization change decoder activations? What semantic or identity information does each digit and codeword represent, for example category, brand, color, size, or style? For RQ, how does the cluster radius change at each digit beyond the mechanical effect of subdividing a cluster, and do later digits refine semantics or mainly separate similar and colliding items? What changes causally when a valid digit or its decoder representation is replaced?

RQ3: How do paradigm-quantizer interactions, SID order, and scale affect recommendation? Are different quantizers favored by auto-regressive and diffusion paradigms? As the number of digits or codebook size increases, do the added codes contribute distinct information or become redundant, and what internal changes accompany the reported performance degradation? For auto-regressive recommendation, what changes when semantic digits are generated fine-to-coarse rather than coarse-to-fine, or in a random order, after accounting for collisions and the training budget?

RQ4: How is the next-two item recommendation computed? Is the second item generated conditionally from the first, or are both items represented and refined as part of a joint plan? When does information about the second item become decodable and causally active, and how do its logits change when the first item’s state or commitment is intervened on? These questions test whether the block-diffusion implementation behaves according to its intended causal structure.

3 Core Experiments and Mechanistic Methods

For every experiment, hooks will save the model’s hidden states and the raw score for every possible SID digit at each decoder layer or diffusion step. A *probe* is a small classifier that checks what information can be read from a hidden state. A sparse autoencoder (SAE) breaks hidden states into possible features. *Activation patching* replaces a hidden state with one from another run, while *ablation* removes a state or feature. These interventions show whether changing an internal component changes the recommendation.

Experiment 1. Map the meaning of every SID digit. At each position, we will measure what digit d adds after digits $1, \dots, d-1$ are already known. We will study classes like category, brand, color, size, style, and also study nearby items in the collaborative data, and item identity. For each codeword, we will list representative items, common metadata values, and nearby codewords that are actually used. We will measure how often each code is used, how uncertain the next digit remains, how much the digit narrows the list of possible items, how often grouped items share an attribute, and whether nearby items in the original embedding remain together.

Tools used. Probes will show when attributes and item identity can first be read from the model. SAEs may find other repeated features, which we will name using the items that activate them most strongly. History patching will replace one past interaction at a time to show which interactions affect each digit. We will also swap a digit with another valid digit or patch its hidden state, then measure changes in digit scores, the final item, its metadata, and its embedding distance. We will assign a meaning to a feature only when changing it produces the expected change in the output.

Experiment 2. Measure RQ cluster geometry at every digit. For residual quantization, items that share the first d digits, $(s_1, \dots, s_d)$, form a depth- d cluster. We use the same pre-quantization item embedding z_i at every depth. For a cluster C , let μ_C be its center. Its root-mean-square radius is

$$r(C) = \sqrt{\frac{1}{|C|} \sum_{i \in C} \|z_i - \mu_C\|_2^2}.$$

This measures how far the items in one cluster lie from its center. The typical radius at depth d is

$$\tilde{r}_d = \text{median} \{r(C) \mid C \in C_d, |C| \geq 2\}.$$

We will show clusters containing one item separately because their radius is always zero, and report the middle 50% of radii and results for different cluster sizes.

We will also measure the overall within-cluster spread:

$$R_d^2 = \frac{\sum_{C \in \mathcal{C}_d} \sum_{i \in C} \|z_i - \mu_C\|_2^2}{\sum_i \|z_i - \mu\|_2^2},$$

where μ is the center of the full catalog. The median radius gives every cluster the same importance. By contrast, R_d^2 gives every item the same importance and tells us what fraction of the catalog’s original variation remains inside the depth- d clusters. Together, the two measures show whether most branches become tighter or whether the result is mainly caused by a few large clusters.

Tools used. Probes will show when the model contains broad prefix information and when it contains exact item information. We will also patch a later-digit state between items that share a prefix, and then between items from nearby prefixes, to test whether later digits change fine details or the broad item type.

Experiment 3. Compare recommendation paradigm and SID quantization. We will test every combination in

$$\{\text{autoregressive, masked diffusion}\} \times \{\text{RQ-VAE, RQ-KMeans, parallel MQ/PSE}\}.$$

The main question is whether the best SID method depends on the recommendation model. RQ may work well with left-to-right generation, while parallel MQ/PSE may work well when diffusion can choose digits in a flexible order. We will compare recommendation quality, error for each digit, whether confidence matches accuracy, how often a valid SID is produced, how quickly the output becomes stable, code usage, collisions, and inference cost.

Tools used. Matched-history activation patching will show how each history position affects each output digit. Raw digit scores will show how the prediction changes across layers or diffusion steps. For diffusion, we will record the order in which digits are chosen, how varied that order is, and whether it changes with the user history. Separate SAEs will first describe features in each model. A paired crosscoder may then compare aligned AR and diffusion states to find features that are shared and features that differ.

Experiment 4. Reverse the AR generation order. We will fine-tune matched AR models using coarse-to-fine order, exact fine-to-coarse order, and random digit orders. The encoder, quantizer, code assignments, starting weights, examples, and beam-search budget will remain fixed. Any extra digit used only to make each item ID unique will remain at the end.

In the main condition, the new order will be used in history SIDs, training targets, evaluation targets, and constrained decoding. A second condition will change only the output order, which separates the effect of output order from the way the history is written.

Tools used. Digit scores and intermediate predictions will show when item identity and attributes become stable. History patching will test whether digit order changes how much the model relies on recent or older interactions. Activation patching will compare matching semantic stages, such as a broad category stage, rather than simply comparing the same token position.

Experiment 5. Analyze scaling and collisions. The supplied scaling study reports that performance can stop improving or may become worse as RQ depth and width increase in its tested AR setting [6]. We will compare settings with similar measured collision rates but different digit counts or codebook sizes. We will record how often codes are used, the uncertainty at each digit, how many items share each prefix, and the makeup of groups whose items share one complete SID.

Tools used. Masking one digit or swapping it with another valid code will test whether extra digits add new information or repeat what earlier digits already provide. For items that share an SID, probes and activation patching will test whether the model still keeps the items separate internally or stores one shared representation for the group. Raw digit scores will show whether longer SIDs fail because of early uncertainty, mistakes that build up, or difficulty choosing between similar items near the end.

Experiment 6. Determine how next-two block diffusion works. We will first check whether the two recommended items use separate information-flow blocks or one shared diffusion block. We will then change the first item while keeping the history, noise, and denoising path fixed and measure how the second item’s scores change.

Tools used. Separate probes for the first and second output positions will show when information about each item can be read. We will replace, delay, or remove the first-item state and test the second item. When the model’s attention rules allow it, we will also test the reverse direction. We will remove the connection that lets one item attend to the other and use history patching to compare the evidence used for the first and second recommendations. Seeing the second item early is not enough to show joint planning; changing one item’s internal state must reliably change the other.

4 Expected Outcome

The project will connect SID geometry to decoder computation and recommendation behavior. Its main outputs will be a semantic atlas of SID digits, RQ radius and refinement curves, controlled paradigm versus quantizer interaction results, causal history-to-digit influence maps, an AR digit-order comparison, and an internal account of scaling, collisions, and next-two-item dependence. The end goal here is to understand how data is passed in either paradigm we test and to understand the internal mechanisms required to perform a recommendation.